Soup - Full-Stack Engineer

Submission note | 11 November 2026

System and architecture

I chose Python 3.12, FastAPI, Pydantic v2, async SQLAlchemy/asyncpg, Alembic and PostgreSQL, with Next.js App Router/React/TypeScript for the interface. I kept all business endpoints and SSE in FastAPI; Next.js serves the browser UI. Docker Compose starts the three services and applies migrations. The demo sends generated metrics through the real ingestion API and database, so it can be explored without running ML training. The repository includes the brief, one-page architecture, API contract, tests and raw output.

Identity, ordering and recovery

I use client-generated UUIDv4 event IDs and a versioned canonical payload to distinguish retries from conflicting data. Each event also carries a metric, logical step, finite value and UTC timestamp. Retrying the same batch_id and content returns its original committed receipt; changed batch or event contents return 409. Identical event IDs across batches reuse their original sequence.

The write transaction locks the authenticated project, then the scoped run. Events, the contiguous per-run seq counter and receipt commit together before a successful response. This makes PostgreSQL the ordering boundary across API processes. The highest (step,seq) determines each current value, so late lower-step events cannot replace newer logical values. If a response is lost, the client retries the same batch to resolve the outcome.

Snapshots use read-only REPEATABLE READ. SSE reads committed rows in pages of 250 from PostgreSQL, releasing sessions before sending. The browser uses fetch/ReadableStream with bearer Authorization, applies contiguous events idempotently and sends Last-Event-ID only after application. Replay covers the snapshot/subscription gap and process restart. Charts retain the last 2,000 events; current values include the full run. Initial chart reconstruction replays from 0; reconnection preserves its applied cursor.

Isolation, rotation and bounds

The bearer token selects project scope for every run lookup, mutation and replay. Only high-entropy token hashes are stored; rotation revokes the old hash transactionally. An independent 1-second watchdog checks existing subscriptions, including blocked replay/send paths. Already-buffered bytes cannot be recalled. Foreign and absent resources share 404 responses; protected responses use no-store. Keys stay in ephemeral browser memory, never URLs, localStorage or public environment variables. Reload requires key entry. Streams have bounded pages, a 5-second send deadline and 100-response/process admission; there is no unbounded subscriber queue.

Consistency and failure evidence

Final automated verification passed 58 PostgreSQL-backed backend tests, 9 real Chromium E2E tests and 5 client tests. Browser tests compare chart points, history, winners, count and cursor with independent SQL after late, out-of-order and duplicate inputs. Cutting frame 11 leaves cursor 10; replay restores every event from 1-30. Other tests cover snapshot gaps, API/DB restarts, stale project frames after key switch, live rotation and rejected cross-project REST/SSE access. PostgreSQL termination, commit rejection, cancellation and lost HTTP receipts exercise rollback and retry. An unread TCP subscriber checks timeout cleanup and replay. Failed runs are included. Two old Phase 2 compile/config logs are missing and have not been recreated.

Load results and verdict

Each local workload used 50 runs producing 10 unique events/s/run in batches of 5, 10 seconds of warm-up and 60 seconds of measurement, plus 300 controlled exact retries. One Uvicorn process ran on Docker Desktop/WSL2 with 16 logical CPUs and 7.61 GiB shared VM memory, on an i5-13450HX laptop. Fixed-window throughput was 498.92 events/s without SSE and 498.17 with 50 SSE subscribers. All-request p95/p99 were 237.56/293.14 ms and 319.31/446.02 ms respectively. Each workload stored all 35,000 planned events and 7,000 receipts, with no HTTP failures/timeouts, missing acknowledged events or duplicate rows; SQL sequences were 1-700 per run and all snapshots matched stored winners. All 50 streams reached 700. One client resumed from 400 and recovered all 55 events acknowledged during its disconnect.

My verdict is SHIP for this measured local profile under the declared 1% tolerance. **The tolerance was self-defined, not specified by Soup. Exact 500 events/s was not achieved in either measured window, and production capacity remains unproven.** These short runs do not establish an indefinite throughput floor or production headroom. Commands, raw results and limitations are in docs/EVIDENCE.md and docs/SUBMISSION_AUDIT.md; reproduction steps are in docs/TESTING.md.

Limits and first production improvement

With 50 SSE consumers, sampled mean API CPU rose from 63% to 104% of one logical core, and observed PG connections reached the pool limit of 20. That shows pressure, but does not establish the saturation point. My first production improvement would be transaction/pool-wait latency and CPU measurements, followed by profiling SSE polling before changing concurrency. Long soaks, many slow subscribers and their aggregate socket buffers, large-history bootstrap, WAN/TLS/proxies, other browsers and disk/power loss remain untested. The exact PostgreSQL COMMIT acknowledgement loss was not injected. Generic errors and local timing/log probes cannot rule out every side channel. Project bootstrap and lost key-rotation response recovery also need a production access policy.

What surprised me and what concerns me

I was surprised that the load generator itself caused the initial throughput problem rather than the backend; I had not expected the test client to be the limiting part. I am still concerned about how the system would behave with many slow SSE clients over a longer period. That scenario has not been properly tested: the existing evidence covers one deliberately constrained slow reader and 50 responsive subscribers for a short workload, not a sustained population of slow clients.

AI disclosure

I used Codex extensively to implement the backend and frontend, review the architecture, generate tests, debug failures, and prepare documentation. I chose the stack, set the scope and UI direction, reviewed the reported results, and guided each phase. Codex executed the automated tests and load benchmarks cited here; I am not presenting those runs as tests I personally executed. Codex revised generated code and tooling after real failures, including SSE cleanup and load-generator overhead. The repository includes the actual output, failed runs, and known limitations.